

Trabalho Prático 2: Arianagram

Após o estrondoso sucesso do álbum *Eternal Sunshine*, Ariana Grande concluiu que as redes sociais convencionais não conseguem mais conectar sua legião de fãs, os *Arianators*, da forma que eles merecem. Para o lançamento de sua nova turnê mundial, ela planeja algo exclusivo: uma rede social minimalista, rápida e livre de algoritmos, onde as interações acontecem em tempo real, diretamente entre pessoas.

Para suportar o tráfego massivo de milhões de fãs postando simultaneamente do mundo todo, o sistema exige um *back-end* robusto e bem projetado. O servidor central precisa gerenciar dezenas de conexões paralelas, garantindo que nenhuma mensagem se perca e que os seguidores recebam as novidades instantaneamente. Ariana, ex-pesquisadora do laboratório LECOM, sabe que os estudantes de Redes de Computadores do professor Flop são os melhores do mundo em programação de sistemas distribuídos e, por isso, convocou você para essa missão.

Sua tarefa é desenvolver o **servidor multithread do Arianagram**. Você implementará o protocolo de comunicação que permite aos fãs postarem pensamentos, seguirem uns aos outros e visualizarem o feed global em tempo real.



1 O Projeto

O objetivo deste trabalho é desenvolver um sistema de rede social baseado no modelo **cliente-servidor**, utilizando comunicação **TCP** e o paradigma **Publish-Subscribe**. O sistema deve permitir que múltiplos usuários se conectem simultaneamente para interagir em tempo real.

O foco central deste TP é o **gerenciamento de múltiplas conexões TCP** e o **correto fluxo de dados entre o cliente e o servidor**. Diferentemente de um simples sistema de requisição-resposta, o servidor aqui age também de forma proativa, encaminhando mensagens a clientes que nem mesmo fizeram uma requisição recente.

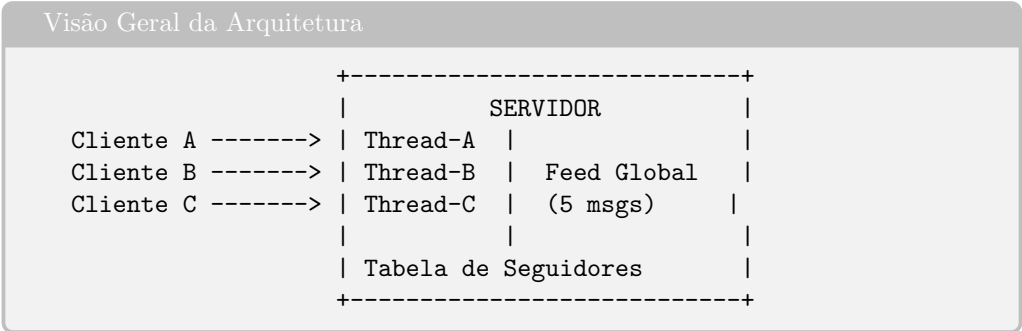
1.1 Arquitetura: Single-Process, Multi-Thread

O servidor deve seguir a arquitetura **Single-Process, Multi-Thread (SPMT)**: um único processo principal (*main loop*) fica aguardando novas conexões em um *socket* de escuta. A cada nova conexão aceita via `accept()`, o servidor cria **uma nova thread POSIX** dedicada exclusivamente àquele cliente.

Essa thread será responsável por:

1. Receber e processar todos os comandos enviados por aquele cliente ao longo de sua sessão.
2. Encaminhar notificações *push* vindas de outros usuários que o cliente segue (veja a Seção 3).
3. Realizar a limpeza de recursos ao final da conexão (veja a Seção 4).

A figura abaixo ilustra a arquitetura do sistema:



1.2 Dinâmica da Rede Social

A interação no Arianagram baseia-se em três operações fundamentais:

- **Postar (MSG_POST)**: O usuário envia uma mensagem de texto. O servidor atribui um ID sequencial global e a armazena no feed, depois notifica imediatamente todos os seguidores ativos via MSG_PUSH.
- **Seguir (MSG_FOLLOW)**: O usuário declara que deseja receber, em tempo real, as postagens de outro perfil. A partir desse momento, qualquer nova postagem do perfil seguido aciona uma notificação automática.
- **Ler Feed (MSG_READ)**: O usuário solicita as últimas mensagens postadas globalmente na rede. O servidor responde com até 5 mensagens históricas, da mais nova para a mais antiga.

2 Protocolo de Comunicação

Toda comunicação entre cliente e servidor é feita através de **mensagens binárias de tamanho fixo**, definidas pela estrutura **Message** abaixo. O uso de tamanho fixo é uma decisão de projeto fundamental: elimina a necessidade de delimitadores de mensagem, assim o servidor sabe exatamente quantos bytes ler.

Listing 1: Estrutura de Mensagem Message

```
1 #define CONTENT_SIZE 140
2 #define USER_SIZE 16
3
4 typedef enum {
5     MSG_CONNECT = 0, /* Cliente -> Servidor: handshake */
6     MSG_POST = 1, /* Cliente -> Servidor: enviar novo post */
7     MSG_FOLLOW = 2, /* Cliente -> Servidor: seguir outro usuario */
8     MSG_READ = 3, /* Cliente -> Servidor: solicitar feed global */
9     MSG_PUSH = 4, /* Servidor -> Cliente: notificacao de seguido */
10    MSG_END = 5 /* Servidor -> Cliente: fim do feed */
11 } MessageType;
12
```

```

13 typedef struct {
14     uint16_t type;           /* Tipo da mensagem (MessageType) */
15     char username[USER_SIZE]; /* Nome do autor (ex: "@ariana") */
16     char content[CONTENT_SIZE]; /* Texto da mensagem ou alvo do FOLLOW */
17     uint32_t msg_id;         /* ID sequencial (preenchido pelo servidor) */
18 } Message;

```

2.1 Campos da Estrutura

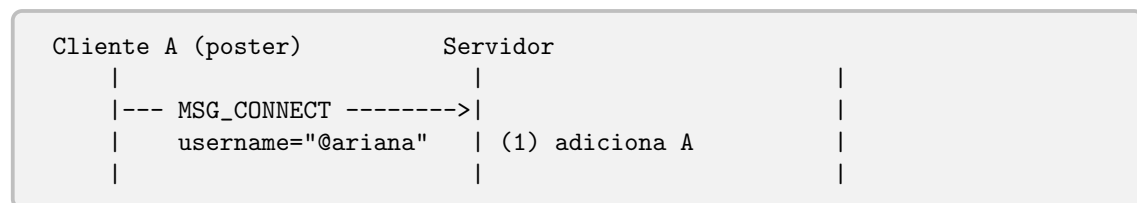
- **type:** Identifica a operação (ver `MessageType`).
- **username:** Nome do usuário **autor** da mensagem. Deve ter no máximo `USER_SIZE - 1` caracteres válidos.
- **content:** No `MSG_POST`, contém o texto da postagem. No `MSG_FOLLOW`, contém o nome do usuário a ser seguido. Em `MSG_READ` e `MSG_PUSH`, o servidor preenche este campo nas respostas.
- **msg_id:** Preenchido **exclusivamente pelo servidor**. O cliente deve enviar este campo como zero nas requisições.

2.2 Fluxos de Mensagens

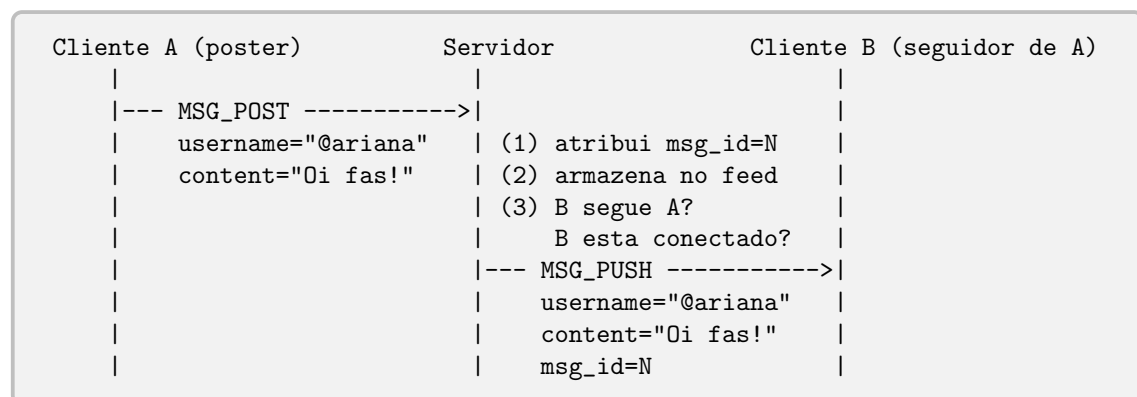
Os diagramas a seguir descrevem os fluxos de comunicação de cada operação.

2.2.1 CONNECT — Handshake de Identificação

Ao estabelecer a conexão TCP, o cliente deve enviar imediatamente um `MSG_CONNECT` com o campo `username` preenchido e `content` e `msg_id` zerados. O servidor, ao receber essa mensagem, registra o nome do cliente e exibe `[CONN] @<usuario> conectou`. Nenhuma resposta é enviada ao cliente.



2.2.2 POST — Publicação com Notificação



Comportamento esperado:

1. O servidor recebe `MSG_POST`, atribui o próximo ID sequencial global e armazena a mensagem no feed circular de 5 posições.
2. O servidor percorre a lista de seguidores de `@ariana`. Para cada seguidor ativo (com socket aberto), envia um `MSG_PUSH` com os dados da postagem.

3. O push é enviado diretamente pela thread do poster (ou pela thread do seguidor, veja a discussão na Seção 3).
4. **Não há resposta de confirmação ao poster.** O MSG_POST é unidirecional do ponto de vista do cliente.

2.2.3 FOLLOW — Inscrição

Cliente B	Servidor
--- MSG_FOLLOW ----->	
username="@b_fan"	(1) registra: B segue A
content="@ariana"	(2) nenhuma resposta enviada

Comportamento esperado: O servidor registra internamente que @b_fan deseja receber pushes de @ariana. Nenhuma mensagem de resposta é enviada ao cliente. A partir desse momento, todo MSG_POST de @ariana desencadeará um MSG_PUSH para @b_fan, *enquanto ele estiver conectado*.

2.2.4 READ — Leitura do Feed Histórico

Cliente C	Servidor
--- MSG_READ ----->	
	(1) busca ultimas N msgs (max 5)
<-- MSG_PUSH (msg mais recente) --	
<-- MSG_PUSH (penultima) -----	
...	
<-- MSG_PUSH (msg mais antiga) ---	
<-- MSG_END -----	(2) sinaliza fim do feed

Comportamento esperado: O servidor responde com até 5 mensagens usando pacotes do tipo MSG_PUSH, enviados da mais recente para a mais antiga. Ao final, independentemente de quantas mensagens foram enviadas, inclusive zero, o servidor envia obrigatoriamente um pacote MSG_END com todos os campos zerados. O cliente usa esse pacote como sinal de fim do feed e retoma o estado normal de espera por comandos do usuário. **O MSG_END é exclusivo da operação MSG_READ e nunca é enviado em outros contextos.**

3 Sistema Publish-Subscribe

O coração do Arianagram é seu mecanismo **Publish-Subscribe (Pub-Sub)**: usuários se inscrevem (*subscribe*) nos posts de outros usuários, e o servidor age como *broker*, entregando (*publish*) as mensagens em tempo real.

Vocês devem pensar nas estruturas de dados que vão utilizar para solucionar os problemas no projeto.

3.1 Fluxo de Notificação Push

Quando a thread do Cliente A processa um MSG_POST, ela deve:

Passo 1. Atribuir ID: Obter e incrementar o ID.

Passo 2. Armazenar no Feed: Inserir a mensagem no feed, sobrescrevendo a mais antiga se o feed estiver cheio.

Passo 3. Percorrer Seguidores: Iterar sobre os seguidores buscando entradas onde `followed == username_do_post`.

Passo 4. Enviar Push: Para cada seguidor encontrado, se o seguidor estiver ativo, montar um Message do tipo MSG_PUSH e enviá-lo ao seguidor.

Atenção: Acesso ao feed global

O Passo 2 envolve uma situação delicada: a Thread-A está escrevendo no **feed** que pertence ao servidor e está acessível para todas as threads do processo. Porém, **você deve garantir que apenas uma thread escreva naquele espaço por vez**. Uma forma simples de fazer isso é manter um *lock* no servidor. Considere as implicações disso no seu design.

4 Gerenciamento de Conexões e Limpeza de Threads

Conexões de rede falham. Clientes fecham o programa sem avisar. Sua implementação **deve lidar com isso corretamente**.

4.1 Detectando o Fechamento de uma Conexão

Quando um cliente fecha a conexão (normalmente ou por crash), o TCP entrega um **FIN** ao servidor. A função `recv()` retornará **0 bytes**, este é o sinal definitivo de que a conexão foi encerrada. Um retorno **negativo** de `recv()` indica um erro (ex: `ECONNRESET`), que também deve ser tratado como encerramento.

4.2 Checklist de Limpeza

Ao encerrar a thread de um cliente, o servidor deve:

- ✓ Chamar `close()` no socket do cliente.
- ✓ Liberar os recursos da thread com `pthread_detach()` (ou `pthread_join()` na thread principal, conforme seu design).

4.3 Dados armazenados no servidor

Se o servidor for encerrado, todas as informações armazenadas devem ser perdidas, o foco do trabalho não é a criação ou a conexão com o banco de dados.

5 Padronização da Saída (Logs)

Para possibilitar a correção automática via `diff`, o padrão de exibição é rigoroso. **Qualquer desvio no formato resultará em perda de pontos**.

5.1 Formatos Obrigatórios

- **Servidor — conexão:**
[CONN] @<usuario> conectou.
- **Cliente — conexão:**
Conectado ao servidor como @<usuario>.
- **Servidor — nova postagem:**
[LOG] @<usuario> posted (ID <id>): «conteudo>
- **Cliente — notificação push recebida:**
[NOTIFICATION] @<usuario>: «conteudo>
- **Cliente — exibição do feed (MSG_READ):**
[FEED] ID <id> | @<usuario>: «conteudo>
- **Servidor — inicialização:**
Aguardando conexoes na porta <porta>.

- **Servidor — desconexão:**
[DISC] @<usuario> desconectou.

Os exemplos a seguir ilustram o comportamento esperado do sistema para cada operação.

5.2 Exemplo 1 — Inicialização e Conexão de Clientes

Terminal do Cliente	Terminal do Servidor
	\$./server v4 51511 Aguardando conexoes na porta 51511.
\$./client 127.0.0.1 51511 @ariana Conectado ao servidor como @ariana.	[CONN] @ariana conectou.
(segundo terminal) \$./client 127.0.0.1 51511 @bfan Conectado ao servidor como @bfan.	[CONN] @bfan conectou.

5.3 Exemplo 2 — Follow

Situação: @bfan decide seguir @ariana. O servidor registra a relação internamente. Nenhuma saída é produzida no terminal do cliente ou do servidor para esta operação.

Terminal do Cliente (@bfan)	Terminal do Servidor
> FOLLOW @ariana	(nenhuma saída)

5.4 Exemplo 3 — Post com Push Notification

Situação: @ariana faz uma postagem. O servidor atribui o ID 1, registra no log e encaminha a notificação para @bfan, que a segue e está conectado.

Terminal de @ariana	Terminal do Servidor	Terminal de @bfan
> POST thank u, next	[LOG] @ariana posted (ID 1): "thank u, next"	[NOTIFICATION] @ariana: "thank u, next"

Situação: @ariana faz uma segunda postagem. Como @bfan ainda está conectado, recebe outra notificação.

Terminal de @ariana	Terminal do Servidor	Terminal de @bfan
> POST positions	[LOG] @ariana posted (ID 2): "positions"	[NOTIFICATION] @ariana: "positions"

5.5 Exemplo 4 — Post sem Seguidores Ativos

Situação: @elsa faz uma postagem, mas nenhum seguidor seu está conectado no momento. O servidor apenas registra o log; nenhum push é enviado.

Terminal de @elsa	Terminal do Servidor
> POST into the unknown	[LOG] @elsa posted (ID 3): "into the unknown"

5.6 Exemplo 5 — Read (Feed Histórico)

Situação: Um novo cliente, @newcomer, conecta-se e solicita o feed. O servidor responde com as últimas mensagens disponíveis (até 5), da mais recente para a mais antiga. O servidor não produz nenhuma saída adicional para esta operação.

Terminal de @newcomer	Terminal do Servidor
> READ	<i>(nenhuma saída)</i>
[FEED] ID 3 @elsa: "into the unknown" [FEED] ID 2 @ariana: "positions" [FEED] ID 1 @ariana: "thank u, next"	

Nota: Se o feed estiver vazio no momento do READ, o cliente não exibe nada. Não existe mensagem de “feed vazio”.

5.7 Exemplo 6 — Desconexão de Cliente

Situação: @bfan encerra sua sessão. O servidor detecta o FIN TCP via `recv() == 0`, registra a desconexão e libera os recursos da thread. A partir desse momento, posts de @ariana não gerarão mais pushes para @bfan.

Terminal de @bfan	Terminal do Servidor
^C (<i>Ctrl+C</i> / <i>encerramento</i>)	[DISC] @bfan desconectou.

Situação: @ariana faz uma nova postagem após a desconexão de @bfan. Nenhum push é enviado.

Terminal de @ariana	Terminal do Servidor	@bfan (desconectado)
> POST god is a woman	[LOG] @ariana posted (ID 4): "god is a woman"	<i>(sem push — offline)</i>

6 Casos de Borda e Comportamento Esperado

Situações ambíguas podem surgir durante a implementação. Nenhum desses casos será avaliado pelo teste automático. O comportamento esperado do servidor para cada uma delas está definido abaixo:

- **Username duplicado:** É permitido que dois clientes se conectem com o mesmo nome de usuário. O servidor **não deve rejeitar** a segunda conexão. Ambos são tratados como instâncias do mesmo perfil: ao enviar um MSG_PUSH para aquele usuário, o servidor encaminha a notificação para **todos os sockets ativos** com aquele nome.
- **MSG_FOLLOW duplicado:** Caso um cliente envie MSG_FOLLOW para o mesmo perfil mais de uma vez, o servidor deve **ignorar silenciosamente** as requisições subsequentes, garantindo que o seguidor apareça **apenas uma vez** na estrutura interna. Isso evita que o mesmo cliente receba pushes duplicados de uma única postagem.
- **MSG_FOLLOW de usuário inexistente:** O servidor **não valida** se o perfil-alvo está conectado ou sequer já existiu. A relação é registrada normalmente. Caso o perfil venha a se conectar e postar no futuro, o push será entregue. Nenhuma resposta é enviada ao cliente e nenhuma saída é produzida no servidor.
- **Auto-follow:** Caso o campo `content` de um MSG_FOLLOW seja idêntico ao `username` do remetente, o servidor deve **ignorar silenciosamente** a requisição. Um usuário não pode seguir a si mesmo.

7 Requisitos Técnicos

- **Linguagem:** C. Não é permitido o uso de C++.
- **Bibliotecas:** Apenas a biblioteca padrão C e a interface POSIX. Bibliotecas externas e *frameworks* são proibidos.

- **Protocolo de Rede:** TCP com suporte tanto a **IPv4** quanto a **IPv6**.
- **Threads:** Obrigatoriamente **pthread**s.
- **IDs Sequenciais:** O servidor deve atribuir IDs de forma estritamente crescente (1, 2, 3...) na ordem em que os posts chegam.
- **Sistema Operacional:** O código deve compilar e executar em **Linux**. **Não** utilize bibliotecas exclusivas de Windows (ex: **winsock**).
- **Interoperabilidade:** Seu servidor deve funcionar corretamente com o cliente de qualquer colega que siga este protocolo.

Atenção: Ordem dos bytes (Endianness)

Os campos numéricos da struct, devem ser transmitidos em **network byte order** (big-endian). Use obrigatoriamente as funções POSIX de conversão ao enviar e ao receber:

- **htons()** / **ntohs()** para **uint16_t**
- **htonl()** / **ntohl()** para **uint32_t**

Sem essas conversões, clientes compilados em máquinas diferentes podem interpretar incorretamente o tipo da mensagem e os IDs, quebrando a interoperabilidade silenciosamente.

8 Avaliação

O trabalho deve ser realizado **individualmente**. A nota será distribuída conforme a Tabela 1. O projeto será corrigido de forma **semi-automática** por uma bateria de testes, cada um verificando uma funcionalidade específica.

Tabela 1: Critérios de Avaliação — TP2

Critério	Peso
Suporte a múltiplas conexões simultâneas (uma thread por cliente)	15%
Handshake inicial	5%
MSG_POST com IDs sequenciais	10%
Pub-Sub: FOLLOW e PUSH em tempo real	25%
MSG_READ com MSG_END	10%
Conformidade do protocolo (big-endian + IPv4/IPv6)	10%
Desconexões e limpeza de threads	5%
Documentação técnica	20%
Total	100%

Será adotada **média harmônica** entre as notas da documentação e da execução. Isso implica que a nota final será **zero** caso uma das partes não seja entregue.

9 Informações sobre a Entrega

A entrega deve ser feita via Moodle, em formato ZIP, com o nome seguindo o padrão:

TP2_NOME_MATRICULA.zip

9.1 Documentação

Cada aluno deve entregar uma documentação em PDF de até **4 páginas** contendo:

- Descrição das estruturas de dados utilizadas no servidor.
- Explicação do fluxo de vida de uma thread (criação, operação, encerramento).

- Discussão dos desafios encontrados no envio de pushes para sockets de outros clientes.
- Decisões de projeto tomadas e suas justificativas.

9.2 Código

- O **Makefile** deve compilar o cliente como `client` e o servidor como `server` na raiz do projeto, sem parâmetros adicionais (`make` apenas).
- O servidor deve ser iniciado como: `./server <v4/v6> <porta>`
- O cliente deve ser iniciado como: `./client <endereço_ip> <porta> <username>`

Atenção: Trabalhos que não compilarem com make não serão corrigidos!

10 Política de Atrasos

O trabalho não será aceito com atraso!

11 Integridade Acadêmica

O Arianagram deve ser **sua** obra. Você pode e deve discutir conceitos com colegas, testar sua implementação contra a deles e tirar dúvidas no fórum, mas o código deve ser 100% autoral.

- **Plágio entre alunos** (cópia de código): nota zero para todos os envolvidos.
- **Uso de código gerado por LLMs** (ChatGPT, Copilot, etc.): nota zero. Ferramentas de análise estrutural de código serão utilizadas na correção.

Seja original. O artista é você — e não uma IA!

12 Materiais de Estudo

1. Capítulos 2 e 3 do livro de sockets disponível no Moodle.
2. *Beej's Guide to Network Programming* — referência indispensável para POSIX sockets.
3. Playlist do Professor Ítalo Cunha — conceitos de sockets e threads em C.
4. `man 3 pthread_create`, `man 2 recv`, `man 3 getaddrinfo` — suas melhores amigas.